Mid Term Project Report

Name: Muhammad Affan Siddiqui

Student Number: 260418005

Lesson Name: Visual Programming

Lesson Number: COM206

INTRODUCTION

Project Title:

Smart Grocery Cart Mobile Application

Research Problem / Problem Statement:

Managing grocery shopping efficiently can be difficult, especially when users need to track selected items, quantities, and total cost at the same time. Traditional methods do not provide a structured way to organize items, which can lead to confusion and inefficient shopping.

This project provides a mobile-based solution that allows users to browse products, filter them by category, and manage their cart dynamically. It improves organization and helps users handle their shopping process more effectively.

Objective of the Project:

The objective of this project is to develop a simple and interactive grocery cart application using Flutter. The system allows users to select products, adjust quantities, and manage their cart in a structured way.

It also demonstrates the use of event-driven programming and dynamic data handling using Dart, where user actions directly update the interface in real time.

Expected Outcome:

The expected outcome is a functional mobile application where users can filter products, add items to a cart, and view updated totals instantly. The application demonstrates how Flutter can be used to build interactive and responsive mobile interfaces.

METHODOLOGY

Approach to Problem Solving:

The application follows an event-driven approach where user interactions directly affect the system behaviour. The design is divided into three main components: product selection, cart management, and total calculation.

The system allows users to select a category, view relevant products, adjust quantities, and add items to the cart. The cart is dynamically updated using a data structure, and all changes are reflected instantly on the user interface.

Input, Processing, Output:

Input:

- User selects category (Fruits, Vegetables, Dairy)
- User increases or decreases product quantity
- User clicks “Add” button
- User interacts with cart (remove or decrease items)

Processing:

- Selected category filters product list
- Items are stored in a list (cart data structure)
- Existing items are updated instead of duplicated
- State is updated using Flutter’s setState()

Output:

- Filtered products displayed
- Cart updates dynamically
- Total price recalculated instantly
- User feedback shown via messages

System Control Flow:

1. Application starts
2. Default category is displayed
3. User selects a category
4. Products are filtered based on category

5. User adjusts quantity
6. User clicks “Add to Cart”
7. Item is added or updated in cart
8. Cart and total are updated
9. User can remove or modify cart items
10. Process continues until checkout

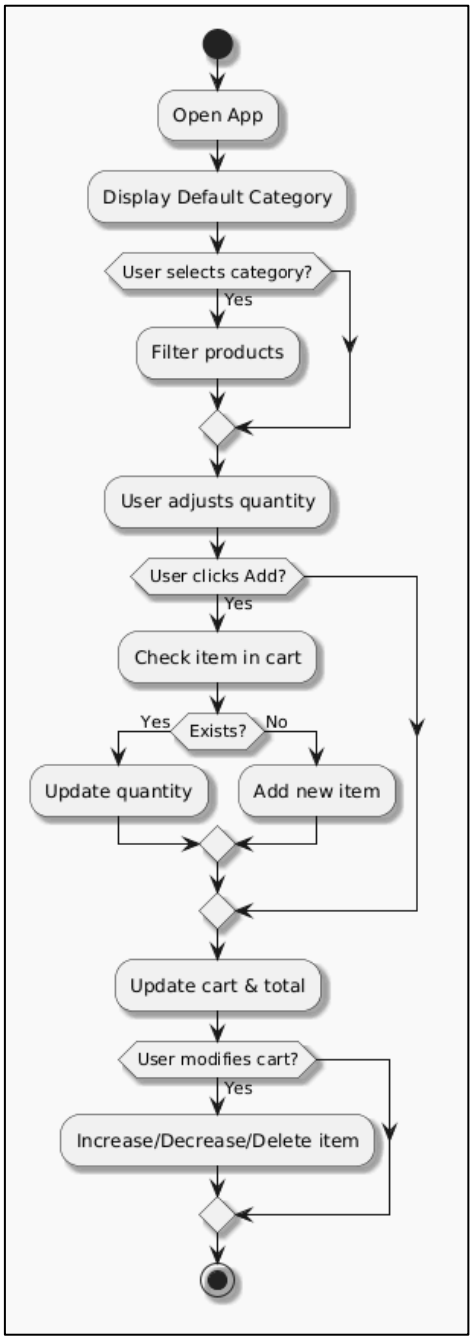

Figure 1: Flowchart of Grocery Cart Application Logic

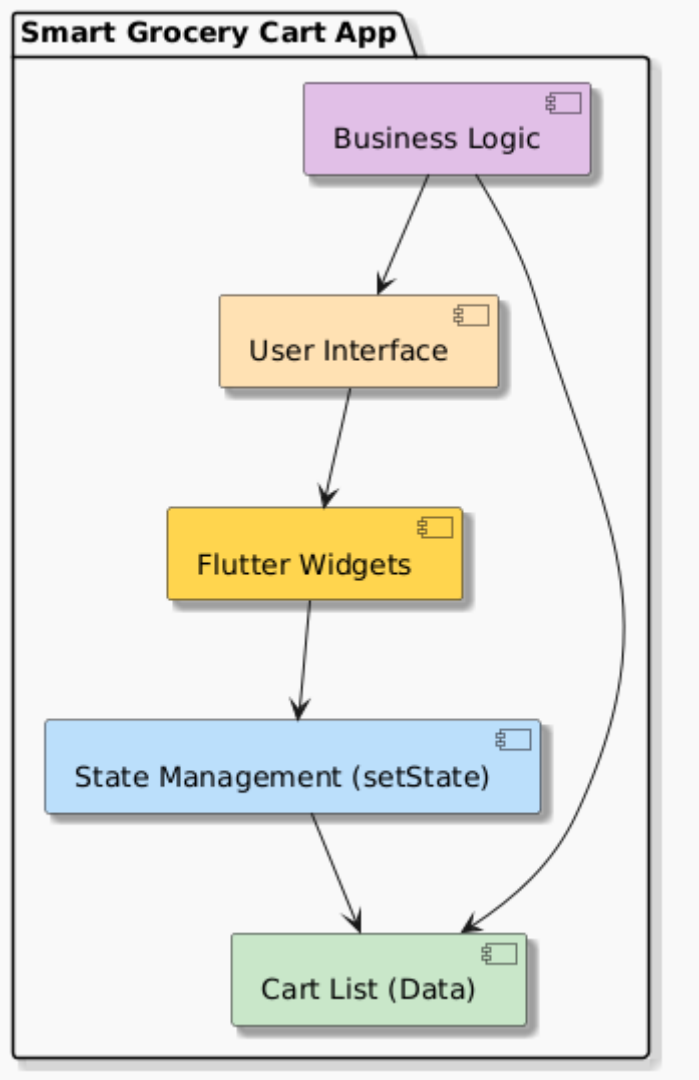

Figure 2: System Structure of Flutter Application

TECHNICAL IMPLEMENTATION

Programming Language:

Dart (Flutter Framework)

Code Example:

```
void addToCart(Map<String, dynamic> product) {
  int index = cart.indexWhere((item) => item["name"] == product["name"]);

  setState(() {
    if (index != -1) {
      cart[index]["qty"] += product["qty"];
    } else {
      cart.add({
        "name": product["name"],
        "price": product["price"],
        "qty": product["qty"]
      });
    }
  });

  showMessage("${product["name"]} added");
}

void removeFromCart(String name) {
  setState(() {
    cart.removeWhere((item) => item["name"] == name);
  });
}

void decreaseCartItem(String name) {
  int index = cart.indexWhere((item) => item["name"] == name);
```

The above snippet presents a portion of the Dart code used in the application, highlighting the core logic of the cart management system. Due to space limitations, only key functions are shown instead of the complete code. The addToCart() function checks whether a product already exists in the cart using a list search; if it exists, the quantity is updated, otherwise a new item is added. The removeFromCart() and decreaseCartItem() functions allow users to manage their cart by removing items or decreasing their quantity. These operations are performed using Flutter's setState() method, ensuring that the user interface updates dynamically in real time. This implementation demonstrates efficient data handling, prevention of duplicate entries, and event-driven interaction within the mobile application.

Explanation:

The application is developed using Flutter, which allows building cross-platform mobile applications using a single codebase. Dart is used as the programming language to implement logic and manage data.

The system uses a list data structure to store cart items. When a user adds a product, the system checks whether the item already exists in the cart. If it does, the quantity is updated; otherwise, a new entry is created. This prevents duplication and ensures efficient data handling.

State management is handled using the setState() method, which updates the user interface whenever there is a change in data. The application also uses widgets such as ListView, ChoiceChip, and Scaffold to create a structured and responsive layout.

User interactions trigger real-time updates, making the application dynamic and interactive. This demonstrates the practical use of event-driven programming in mobile application development.

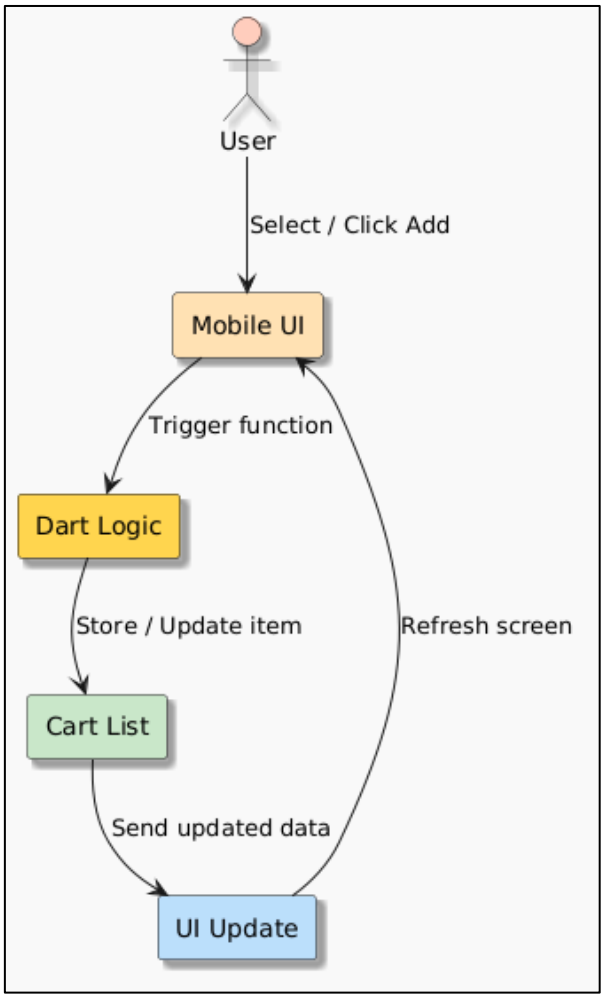

Figure 3: Data Flow in Grocery Cart Application